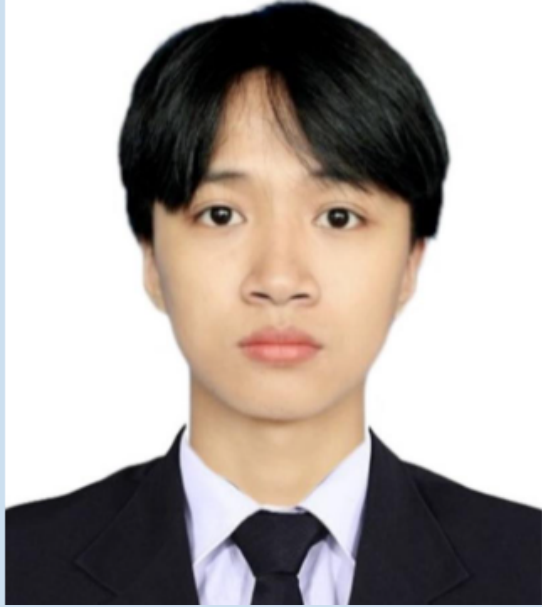




NGUYEN DUC QUANG

Java Backend Developer Intern

SUMMARY

Detail-oriented Information Technology student specialising in Java Backend Development. Strong foundation in Java Core, Spring Boot, Spring Security, and Hibernate/JPA. Hands-on experience in building asynchronous trading engines, optimising database performance, and designing secure RESTful APIs. Ready to contribute clean code practices and problem-solving skills to a Backend Java Developer Intern position.

DETAILS

✉ nguyenducquang19092004@gmail.com

📍 Ha Noi, Việt Nam

☎ +84382489793

EDUCATION

Engineering Degree, Hanoi University of Civil Engineering, Ha Noi

GPA: 3.05
Expected graduation: 2027

SKILLS

- Programming Languages: Java (Core/17+)
- Backend Technologies: Spring Boot, Spring Data JPA, Hibernate ORM, Spring Security, JWT Authentication, RESTful API, Validation & Exception Handling, Event-Driven Architecture (Basic)
- Databases: PostgreSQL, MySQL, SQL (Indexing & Optimization)

EXPERIENCE

Core Trading Backend Engine, Personal Project (Production-Ready Practice)

- Context: A high-throughput backend system simulating an electronic stock exchange core. The system handles complex financial workflows including real-time user wallet balances, stock order matching, multi-threaded market tick simulation, and automated candlestick charts (OHLCV) aggregation.
- Tech Stack: Java 17, Spring Boot 3.x, Spring Data JPA, Hibernate, Spring Security, JWT, PostgreSQL, Docker, Docker Compose, Swagger/OpenAPI.
- Architecture & Design Patterns: Developed strictly under 3-Tier Layered Architecture to enforce separation of concerns and maintainability. Applied Event-Driven Architecture (Pub/Sub) using ApplicationEventPublisher to cleanly decouple core trading modules from third-party notification services (Email/Alerts). Integrated the BFF (Backend For Frontend) Pattern via a unified /prepare-order endpoint, consolidating stock, wallet, and asset states into a single round-trip to prevent API waterfall bottlenecks on client apps. Implemented Observer Pattern for pricing alerts and utilized Java Records to guarantee immutable, thread-safe data transfer objects (DTOs).

- DevOps & Tools: Docker, Docker Compose, Git/GitHub, Postman, Swagger/OpenAP
- Core Concepts: Object-Oriented Programming (OOP), SOLID Principles, Clean Code, 3-Tier Layered Architecture, Design Patterns (Observer, DTO)
- Performance & Database Optimization: Eliminated N+1 query vulnerabilities by strictly leveraging FetchType.LAZY and custom JPQL fetch joins. Leveraged DTO Projection combined with Correlated Scalar Subqueries to aggregate complex data from 3 separate tables (stocks, wallets, assets) in a single SQL execution scan, reducing API response time to ~0.01 seconds. Designed a time-series database structure with Composite Indexes and strategic Denormalization on high-frequency tables (market_ticks, candles) to dramatically accelerate chart rendering. Enforced full data consistency across multi-threaded environments by managing transaction boundaries with @TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT).
- Deployment & Modern Practices: Packaged the system utilizing Docker with Multi-stage builds to minimize footprint and deployed via Docker Compose to isolate the core engine and PostgreSQL within a secure virtual private network.